Augmenting Datasets with Generated Data

Methods and Evaluation

Rebecca Ansell

Georgetown University • PhD Student

CAPWIC 2026

Workshop Agenda

01 **The Limited Data Problem**
Why 50 labeled examples aren't enough

02 **Generating Synthetic Data**
Three prompting methods using local LLMs

03 **Evaluating Your Results**
Distribution, performance, and diversity metrics

04 **The Full Pipeline**
A reusable augmentation workflow

05 **Code Walkthrough**
Hands-on notebook demo and exercises

The Limited Data Problem

A common problem:

- Labeling data is expensive and time-consuming
- Domain experts are scarce
- Often the bottleneck in a research process

Which results in...

- Models overfit to small training sets
- Poor generalization to unseen data

What if you could ask an LLM to **generate more?**

Use local models via Ollama to create synthetic training data that augments your limited labeled set

50 examples → 200+

Our Running Example

Scenario

You're building a **customer support intent classifier** that routes incoming messages to the right team. You need to classify messages into 5 categories.

The constraint

- You only have **10 labeled examples per class** (50 total)
- Not enough to train a reliable classifier
- Getting more labeled data would take weeks

The approach

Use a local LLM (**Qwen 2.5 via Ollama**) to generate synthetic training examples, then measure whether they actually help.

The Five Classes

Customer Support Intent Classification — 5 classes, 10 examples each

billing

"Why was I charged twice this month?"

technical support

"App keeps crashing on login"

account

"How do I update my email?"

returns

"I need to return a defective item"

general inquiry

"What are your store hours?"

Goal: Can we generate synthetic data to boost classifier accuracy?

Three Generation Methods

Increasing complexity and control →

1

Simple Prompting

Plain instruction to the LLM

+ Fast, minimal setup
– Less variety, may drift

2

Few-Shot Prompting

Real examples as demonstrations

+ More realistic, grounded
– Requires quality seed data

3

Controlled Generation

Scenario-driven sub-cases

+ Best edge case coverage
– Needs domain knowledge

Method 1: Simple Prompting

How It Works

- Direct instruction to the LLM — no examples provided
- Higher temperature (0.8) for variety

+ Fast, minimal setup, no seed data needed
– May generate generic or off-distribution text

Method 2: Few-Shot Prompting

How It Works

- Include 3 real examples in the prompt as demonstrations
- Model learns tone, length, and style from your data

+ More realistic, on-distribution outputs

– Requires quality seed examples

Method 3: Controlled Generation

How It Works

- Define sub-scenarios per class, generate 2 samples each
- Targets edge cases the model wouldn't produce on its own

+ Best diversity and edge-case coverage

– Requires domain knowledge to define scenarios

Evaluating Synthetic Data

Three axes to measure whether generated data is actually useful

1

Distribution Alignment

Does generated data look like real data?

TF-IDF centroid cosine similarity

2

Downstream Performance

Does augmentation improve the classifier?

Accuracy comparison across conditions

3

Diversity Metrics

Does it add new information?

Type-token ratio, unique bigrams

Ensuring Distribution Fit

Measurements

- Centroid similarity — is generated text on-topic for each class?
- Pairwise similarity — is it diverse, or just repeating itself?
- Downstream accuracy — does adding it actually help the classifier?

Considerations

- Label leakage between classes
- Model artifacts and repetitive phrasing
- Always validate with held-out test data

Distribution & Diversity Metrics

Distribution Similarity

- TF-IDF centroid cosine similarity per class
- High centroid sim = on-topic; moderate pairwise sim = not copies

Diversity Metrics

- Type-token ratio (vocabulary richness)
- Unique bigrams (phrase variety)
- Average length (structural consistency)

Downstream Task Evaluation

Does adding synthetic data actually improve your model?

The approach

- Train your model with and without the augmented data
- Evaluate on a held-out test set the model has never seen
- Compare each generation method individually and combined

What to look for

- Accuracy goes up → augmentation is helping
- Accuracy drops → generated data is hurting, revisit your method

The Full Pipeline



Key Takeaways

- LLMs can generate useful training data **locally and for free**
- More structure in your prompt = **better output**
- Always evaluate **before** you trust synthetic data
- Combine methods for **better coverage**

Questions?

before we move to the practical example

Let's Code!



Scan to open the notebook
https://colab.research.google.com/drive/1dkKG_5-LXHZVmqpxbHNIYaQ_5xpaWyJN?authuser=1